DESIGN AND ANALYSIS OF ALGORITHMS

LAB 01 | Programming Foundations in C++ or Python

Zhengzhou University • BS Software Engineering • Individual Lab • 120 minutes

Choose ONE language: C++ or Python. Complete the same exercises in your chosen language. No sorting algorithms are required.

1 Learning outcomes

Use variables, input/output, swapping, simple if, if/else, if/else if, loops, arrays/lists and sequential search. Test programs using different inputs.

2 Setup and first program

Create a folder named Algorithms_Lab01_StudentID. You may use a local C++17 compiler or Python 3. For C++, an online compiler is also available at https://www.programiz.com/cpp-programming/online-compiler/ . Save C++ files as .cpp and Python files as .py.

At the top of EVERY program, write your name and student ID in comments, using // for C++ or # for Python.

C++

```
// Name: Your Name
// Student ID: Your ID
#include <iostream>
using namespace std;
int main() {
  cout << "Welcome to Algorithms Lab!";
  return 0;
}
```

Python

```
# Name: Your Name
# Student ID: Your ID
print("Welcome to Algorithms Lab!")
```

Run the program. Expected output: Welcome to Algorithms Lab!

3 Variables, input and arithmetic

Read two integers and print their sum and product.

C++

```
#include <iostream>
using namespace std;
int main() {
  int a, b;
  cin >> a >> b;
  cout << "Sum: " << a+b << "\n";
  cout << "Product: " << a*b;
```

Python

```
a = int(input())
b = int(input())
print("Sum:", a+b)
print("Product:", a*b)
```

```
  return 0;
}
```

Test input: 4 and 6 (on separate lines). Expected: Sum: 10; Product: 24.

Practice: Change the program to also print the difference a - b.

4 Swap two numbers

Swapping means exchanging the values of two variables. Here we use a temporary variable.

C++

```
#include <iostream>
using namespace std;
int main() {
  int a, b, temp;
  cin >> a >> b;
  temp = a;
  a = b;
  b = temp;
  cout << a << " " << b;
  return 0;
}
```

Python

```
a = int(input())
b = int(input())
temp = a
a = b
b = temp
print(a, b)
```

Test input: 5 and 9. Expected output: 9 5.

Practice: Run the swap program using 12 and 20. Write down the output.

5 Conditional statements

5.1 Simple if

A simple if executes a statement only when its condition is true.

C++

```
int marks; cin >> marks;
if (marks == 100) {
  cout << "Perfect score";
}
```

Python

```
marks = int(input())
if marks == 100:
    print("Perfect score")
```

Test: 100 prints Perfect score; 75 prints nothing. C++ snippets in this section go inside main() after including iostream and using namespace std.

5.2 if / else: Pass or Fail

Assume marks range from 0 to 100. A mark of 60 or above is Pass; otherwise Fail.

C++ **Python**

```cpp
int marks; cin >> marks;
if (marks >= 60) {
  cout << "Pass";
} else {
  cout << "Fail";
}
```

```python
marks = int(input())
if marks >= 60:
    print("Pass")
else:
    print("Fail")
```

Test: 75 → Pass; 45 → Fail; 60 → Pass.

Practice: Modify the program so the passing mark is 50. Test marks 49 and 50.

5.3 if / else if / else: Grades A to F

Use the following grading scale for this exercise: A = 90–100, B = 80–89, C = 70–79, D = 60–69, E = 50–59, F = 0–49. Reject marks outside 0–100.

C++

```cpp
int m; cin >> m;
if (m < 0 || m > 100) cout << "Invalid";
else if (m >= 90) cout << "A";
else if (m >= 80) cout << "B";
else if (m >= 70) cout << "C";
else if (m >= 60) cout << "D";
else if (m >= 50) cout << "E";
else cout << "F";
```

Python

```python
m = int(input())
if m < 0 or m > 100:
    print("Invalid")
elif m >= 90:
    print("A")
elif m >= 80:
    print("B")
elif m >= 70:
    print("C")
elif m >= 60:
    print("D")
elif m >= 50:
    print("E")
else:
    print("F")
```

Test: 95 → A; 82 → B; 73 → C; 65 → D; 54 → E; 42 → F; 110 → Invalid.

Practice: Enter your own test marks for each grade A–F and one invalid mark. Record the results.

6 Loops: for and while

A for loop is useful for a known number of repetitions. A while loop continues while a condition remains true.

C++

```cpp
for (int i=1; i<=5; i++) {
  cout << i << " ";
}
// while example
int j=1;
while (j<=3) {
  cout << j << " ";
  j++;
}
```

Python

```python
for i in range(1, 6):
    print(i, end=" ")
# while example
j = 1
while j <= 3:
    print(j, end=" ")
    j += 1
```

Expected: for loop prints 1 2 3 4 5; while loop prints 1 2 3. Run these examples separately if you want separate output lines.

Practice: Print even numbers 2, 4, 6, 8, 10 using a loop. Then calculate the sum 1+2+3+4+5 (15).

7 Arrays and Python lists

Arrays/lists store multiple values. Index 0 refers to the first element.

C++

```
int a[]={2,4,6};
int total=0;
for (int i=0;i<3;i++) total+=a[i];
cout << total;
```

Python

```
a = [2, 4, 6]
total = 0
for value in a:
    total += value
print(total)
```

Expected output: 12.

Practice: For [8, 3, 11, 6, 2], print the first element, third element and total. Expected: 8, 11, 30.

8 Sequential (linear) search

Search one item at a time. Print the index of the first match, or -1 if the value is absent.

C++

```
int a[]={4,9,2,7,5};
int key=7, pos=-1;
for(int i=0;i<5;i++){
  if(a[i]==key){pos=i;break;}
}
cout << pos;
```

Python

```
a = [4, 9, 2, 7, 5]
key = 7
pos = -1
for i in range(len(a)):
    if a[i] == key:
        pos = i
        break
print(pos)
```

For key 7, expected output is 3. For key 10, expected output is -1.

Practice: Ask the user for a target and search [3, 6, 1, 8]. Test 6 → 1 and 5 → -1.

9 Optional challenge: binary search

Binary search requires sorted data. Try searching for 12 in [2, 4, 6, 8, 10, 12, 14] by checking the middle element and discarding half each time. No sorting implementation is needed.

10 Files to submit

Submit the .cpp OR .py source files for your completed practice programs. Put your name and student ID in a comment at the top of every file. Use clear filenames, for example swap.cpp,

pass_fail.py, grades.cpp, or linear_search.py. Test that each file runs before submitting. Submit through the class monitor.